CHAPTER 2

ARRAYS

CHAPTER ESSENCE AT A GLANCE:
Arrays allocate contiguous homogeneous blocks of memory to enable deterministic indexing, constant-time address computation, and structured sequential traversal algorithms.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Contiguous Allocation and Base Addressing:** An array reserves adjacent linear memory cells where the location of any element is computed in constant time using the base address, the element subscript, and the byte size of the fundamental data type.

• **One-Dimensional Array Traversal and Bounds:** Single-dimensional arrays store sequential values accessible via zero-based indexing, requiring algorithmic bounds discipline because C++ does not perform runtime subscript range checking.

• **Two-Dimensional Matrix Mapping:** Two-dimensional arrays represent tabular grid structures mapped into linear physical memory in Row-Major Order, computing addresses through row offsets scaled by column dimensions.

• **Null-Terminated Character Strings:** Textual strings in C++ are represented as character arrays terminated by the invisible null character sentinel, enabling library functions in cstring to measure length, copy, concatenate, and compare.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{1D Array Address Formula} = \text{Loc}(A[i]) = \text{Base}(A) + i \cdot W \quad (W = \text{sizeof(Type)},\ i \geq 0)$$

Linear Array Element Address Calculation

$$\text{2D Row-Major Address} = \text{Loc}(A[i][j]) = \text{Base}(A) + (i \cdot C + j) \cdot W \quad (C = \text{Total Columns})$$

Two-Dimensional Row-Major Mapping

$$\text{Linear Search Complexity} = \frac{T_{\text{best}} = 1 \text{ Comparison}, \quad T_{\text{worst}} = n \text{ Comparisons}}{T_{\text{avg}} = \frac{n+1}{2} \text{ Comparisons}}$$

Sequential Search Operational Latency

$$\text{String Storage Model} = \text{Allocated Bytes} \geq \text{String Length} + 1 \quad (\text{Last Byte} = \text{'\textbackslash 0'})$$

Null-Terminated String Allocation

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** C++ does not provide automatic array index bounds checking; accessing array indices beyond the allocated dimension causes memory corruption and unpredictable runtime faults.

• **Alert:** Always allocate at least n plus 1 characters for a string of length n to accommodate the terminating null character sentinel; omitting it leads to buffer overrun.

• **Alert:** When passing a two-dimensional array to a function, the column dimension must always be explicitly stated in the parameter declaration.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. Define an array in C++. State two distinct advantages of using arrays over individual scalar variables. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Definition: A collection of homogeneous (same data type) elements stored in contiguous memory locations and referenced by a common variable name and subscript indices. **[1 Mark]**

• Advantages: Eliminates declaring multiple distinct variable names for related data items, and allows rapid iterative traversal using loops with index variables. **[1 Mark]**

2. Differentiate between an integer array and a character array (string) in C++ regarding their termination and memory allocation. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Integer Array: Stores numeric integer values in contiguous memory cells without any special terminal indicator; size is governed strictly by the declared dimension. **[1 Mark]**

• Character Array (String): Stores textual characters terminated by an explicit null character sentinel ('\0') requiring one additional byte of memory beyond text length. **[1 Mark]**

3. Explain the Linear Search algorithm with its step-by-step procedure and write its best-case and worst-case time complexities. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Algorithm Logic: Sequential search starts at the zeroth element and compares each array element consecutively with the target search key until a match is found or the array ends. **[1.5 Marks]**

• Best-Case Complexity: Key found at the very first position (index 0), requiring exactly 1 comparison (O(1)). **[0.75 Mark]**

• Worst-Case Complexity: Key located at the last position (index n-1) or absent from the array, requiring n comparisons (O(n)). **[0.75 Mark]**

4. An array int Marks[10] is stored with a base address of 2000 in memory. Calculate the memory address of Marks[6] assuming each integer occupies 4 bytes. Show the formula. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Formula Specification: Address of A[i] = Base Address + (i * Element Size in Bytes). Here Base = 2000, i = 6, Size W = 4 bytes. **[1 Mark]**

• Calculation: Address of Marks[6] = 2000 + (6 * 4) = 2000 + 24 = 2024. **[1 Mark]**

• Result Box: The calculated physical memory address of Marks[6] is 2024. **[1 Mark]**

5. Write a C++ function to find and display the largest element from an integer array of size N. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Function Header: void findMax(int arr[], int n) { int maxVal = arr[0]; **[1 Mark]**

• Comparison Loop: for (int i = 1; i < n; i++) { if (arr[i] > maxVal) { maxVal = arr[i]; } } **[1 Mark]**

• Display Output: cout << 'Largest element is: ' << maxVal << endl; } **[1 Mark]**

6. Describe four standard string manipulation functions available in the <cstring> header file with their syntax and purpose. [Score: 3 Marks]

Answer & Key Valuation Steps:

• strlen(s): Computes and returns the exact number of characters in string s excluding the terminating null character. **[0.75 Mark]**

• strcpy(dest, src): Copies the contents of source string src including the null character into destination string dest. **[0.75 Mark]**

• strcat(dest, src): Concatenates source string src onto the end of destination string dest, overwriting dest's null character. **[0.75 Mark]**

• strcmp(s1, s2): Lexicographically compares two strings; returns 0 if identical, positive if s1 > s2, and negative if s1 < s2. **[0.75 Mark]**

7. Write a complete C++ program to read N integers into an array, input a search key, perform linear search, and print whether the key was found along with its index position. [Score: 4 Marks]

Answer & Key Valuation Steps:

• Program Setup: Include iostream, using namespace std, define main(), and declare int arr[50], n, key, i, found = 0, pos = -1. **[1 Mark]**

• Data Input: Input size n, then read n elements using a for loop: for (i = 0; i < n; i++) cin >> arr[i]; then input search key. **[1 Mark]**

• Linear Search Loop: for (i = 0; i < n; i++) { if (arr[i] == key) { found = 1; pos = i; break; } } **[1 Mark]**

• Result Reporting: if (found == 1) cout << 'Found at index: ' << pos; else cout << 'Element not found in array'; return 0. **[1 Mark]**